Documentación

- Documentación
  - Generación de documentación
  - Ejemplos de documentación:
  - Herramientas para documentar
  - JSDoc
    - Bloques de comentarios
    - Etiquetas JsDoc
    - Documentando una función
      - Author
      - Parámetros
      - Valor de retorno
      - Errores
      - Versionado
      - Ejemplo de invocación
    - Documentando una clase
      - Clase sin documentación
      - Clase documentada
  - Documentar un manejador de ruta
  - Documentar o No Documentar?
    - Ventajas
    - Desventajas

Generación de documentación

La generación de documentación generalmente es considerada una tarea tediosa, aburrida e incluso es muy frecuente que no se realize.

Cuando estamos trabajando en código de otros programadores, es una perdida de tiempo analizar el código para tratar de descifrar la lógica y funcionalidades.

Aquí es donde se vuelven muy importante aspectos como:

- Una adecuada estructura del código
- La nomenclatura usada para el nombre de funciones, procedimientos y variables.
- Una buena documentación del código

Ejemplos de documentación:

- **Guías "How To" y tutoriales**
- **Archivos `README`**
- **Documentación en diversos formatos como PDF y HTML**

Si vamos a realizar una guía o tutoriales estaremos obligados a actualizarlos manualmente cada que hagamos algún cambio y esto puede volverse tedioso.
Un archivo README siempre debe estar actualizado, pero este proceso también es manual.

Herramientas para documentar

En Javascript para ayudarnos en el proceso de generación de documentación de nuestro código contamos con herramientas como JsDoc. Todos los lenguajes tienen herramientas que automatizan el proceso de generar la documentación:

Lenguaje	Herramienta
Java	Javadoc
Php	PhpDocumentor
Python	Pdoc
Javasxript	JsDoc

JSDoc

JsDoc es:

- Es un **lenguaje para añadir documentación** a nuestro código fuente Javascript.
- Es una **herramienta para generarla**

Bloques de comentarios

Antes de cada función deberemos agregar bloquest de comentarios.
Estos comentarios deben tener el siguiente formato:

```
/**
    Comentarios JsDoc
*/
```

> Al usar JsDoc el auto completado de los editores de texto e IDES
> mejorará ya que se tomarán en cuenta las anotaciones en estos comentarios

Etiquetas JsDoc

Dentro de los comentarios usaremos etiquetas, las etiquetas nos sirven para describir nuestro código, comienzan con `@`

- `@Author` Que persona elaboró el código
- `@return` Valor de retorno de una función
- `@throws` Excepción que puede lanzar una función
- `@deprecated` Indica que la función es obsoleta
- `@param` Parámetros de la función

Documentando una función

Agregando una descripción de la función:

```
/**
*   Función que multiplica un número por si mismo
*/
function multi(valor) {
    return valor*valor;
}
```

Author

Agregando el nombre del desarrollador:

```
/**
*   Función que calcula el cuadrado de un número
*   @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
*/
function multi(valor) {
    return valor*valor;
}
```

Parámetros

Agregando la descripción de los parámetros que recibe una función:

```
/**
*   Función que calcula el cuadrado de un número
*   @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
*   @param {Number} valor - Valor que se va elevar
*/
function multi(valor) {
    return valor*valor;
}
```

Valor de retorno

Agregando el valor de retorno:

```
/**
*   Función que calcula el cuadrado de un número
*   @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
*   @param {Number} valor - Valor que se va elevar
*   @returns {Number} El número elevado al cuadrado
*/
function cuadrado(valor) {
    return valor*valor;
}
```

Errores

Agregando los errores que puede regresar:

```
/**
*   Función que calcula el cuadrado de un número
*   @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
*   @param {Number} valor - Valor que se va elevar
*   @returns {Number} El número elevado al cuadrado
*   @throws {TypeError} Si valor no es un numero
*/
function cuadrado(valor) {
    return valor*valor;
}
```

Versionado

Podemos indicar la versión y desde que versión existe

```
/**
*   Función que calcula el cuadrado de un número
*   @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
*   @param {Number} valor - Valor que se va elevar
*   @returns {Number} El número elevado al cuadrado
*   @throws {TypeError} Si valor no es un numero
*   @version 3.1
*   @since   2.2
*/
function cuadrado(valor) {
    return valor*valor;
}
```

Ejemplo de invocación

Podemos agegar ejemplo(s) de invocacion(es) (puede haber mas de uno)

```
/**
*   Función que calcula el cuadrado de un número
*   @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
*   @param {Number} valor - Valor que se va elevar
*   @returns {Number} El número elevado al cuadrado
*   @throws {TypeError} Si valor no es un numero
*   @version 3.1
*   @since   2.2
*   @example
*   // Caso de uso
*   cuadrado(4); // → 16
*/
function cuadrado(valor) {
    return valor*valor;
}
```

Documentando una clase

Clase sin documentación

Ejemplo de una clase en Javascript sin documentar:

```
class Transporte {

  constructor(marca, color, capacidad) {
    this.marca     = marca;
    this.color     = color;
    this.capacidad = capacidad;
    this.encendido = false;
  }

  arrancar() {
    if (this.encendido) {
      console.log(`${this.marca} ya está encendido.`);
      return;
    }
    this.encendido = true;
    console.log(`${this.marca} ha arrancado.`);
  }

  apagar() {
    if (!this.encendido) {
      console.log(`${this.marca} ya está apagado.`);
      return;
    }
    this.encendido = false;
    console.log(`${this.marca} se ha apagado.`);
  }
}
```

Clase documentada

Agregando bloques de comentarios a la clase:

```
/**
* Clase Transporte
*
* @class
* @author Gerardo Pineda <gerardo.pz@nlaredo.tecnm.mx>
* @version 1.0.0
* @since 2025-01
*
* @example
* const auto = new Transporte('Toyota', 'rojo', 5);
*/
class Transporte {
  /**
  * Crea una instancia de Transporte.
  * @constructor
  * @param {string} marca     - Marca del vehículo
  * @param {string} color     - Color del vehículo
  * @param {number} capacidad - Número de pasajeros
  *
  * @property {string}  marca     - Marca del vehículo
  * @property {string}  color     - Color del vehículo
  * @property {number}  capacidad - Número de pasajeros
  * @property {boolean} encendido - Estado (default: false)
  */
  constructor(marca, color, capacidad) {
    this.marca     = marca;
    this.color     = color;
    this.capacidad = capacidad;
    this.encendido = false;

  /**
   * Método para encender, si ya está encendido, no hace nada.
   * @method
   * @returns {void}
   * @example
   * auto.arrancar();
   */
  arrancar() {
    if (this.encendido) {
      console.log(`${this.marca} ya está encendido.`);
      return;
    }
    this.encendido = true;
    console.log(`${this.marca} ha arrancado.`);
  }

  /**
   * Metodo para apagar. Si ya está apagado, no hace nada.
   * @method
   * @returns {void}
   * @example
   * auto.apagar();
   */
  apagar() {
    if (!this.encendido) {
      console.log(`${this.marca} ya está apagado.`);
      return;
    }
    this.encendido = false;
    console.log(`${this.marca} se ha apagado.`);
  }
}
```

Documentar un manejador de ruta

Para documentar los manejadores de ruta de un servidor usaremos la etiqueta **`@swagger`**
Deberemos describir el objeto OpenApi de esa ruta usando el formato YAML en el bloque de comemtarios.

```
/**
* @swagger
* /usuario/{id}:
*   get:
*     tags:
*       - Usuario
*     summary: Obtener Usuarios
*     description: Devuelve arreglo de usuarios
*     produces:
*       - application/json
*     parameters:
*       - in: path
*         name: id
*         description: ID del usuario
*     responses:
*       200:
*         description: Regresa un arreglo
*         type: json
*/
app.get('/usuario/:id', async (req,res) => {
    const connection = await mysql.createConnection({host:HOST, user:USER, database:DATABASE, port:PORT, password:PASSWORD});
    const [rows, fields] = await connection.execute('SELECT * FROM `cusuario` WHERE `idUsuario` = ?', [req.params.id]);
    if (rows.length==0) {
        res.json({registros:"No se encontro usuario"});
    } else {
        res.json(rows);
    }
});
```

Documentar o No Documentar?

Si observamos los ejemplos anteriores podemos ver claramente que el código crece con los bloques de comentarios pero debemos tomar en cuenta que estos son ignorados por el compilador, además de que los empaquetadores como Vite, Webpack y demas los eliminan para hacer el bundle, asi que el rendimiento no se ve afectado.

Ventajas

Autocompletado y comprobacion de tipos
Los IDES y editores de código leen las etiquetas para mejorar el autocompletado
Incluso sugieren los tipos de datos corretos cuando escribimos el código.

Mejora trabajo en equipo
Nuevos desarrolladores que se integren a un proyecto lo entenderan mas fácil, incluso el mismo desarrollador que lo elabor≤ puede en unos meses o años no recordar que hace una función

Generación automática de manuales
Las herramientas que generan la documentación pueden productir sitios en HTML o formatos como PDF.
de manera automática. muchas librerías de código publicadas en el registro npm están documentadas con JsDoc.

Desventajas

Mantenimiento
Puede llegar a ser tedioso crear la docuimentación, además de que cualquier modificación al código deberemos actualizarla en los bloques de comentario también,